Experiment - 5

Student Name: Reyansh Goswami **UID:** 23BCS11136
Branch: BE-CSE **Section/Group:** KRG_2B
Semester: 6th **Date of Performance:** 25/2/26
Subject Name: System Design
Subject Code: 23CSH-314

Aim:

To design and analyze a scalable real-time messenger application that allows users to communicate via one-to-one and group messaging with support for text and media, ensuring high availability, low latency, and reliability at large scale.

Objectives:

1. To understand system design principles for large-scale messaging systems.
2. To design APIs for user management and messaging services.
3. To identify functional and non-functional requirements.
4. To apply CAP theorem concepts in distributed systems.
5. To design a high-availability, low-latency architecture.

Procedure-

1. Identify functional requirements of a messenger application.
2. Define non-functional requirements such as scalability, latency, and availability.
3. Analyze CAP theorem trade-offs.
4. Identify core entities required for system implementation.
5. Design the system architecture using Draw.io.
6. Validate the design against real-time message transmission behavior.

Functional Requirements -

- **User Management**

1. User should be able to register.
2. User should be able to login/logout.
3. User should be able to update profile information.

Messaging

1. System should support one-to-one messaging.
2. System should support group messaging.
3. System should support text messages.
4. System should support media messages (images, videos, documents).
5. System should preserve message history.
6. System should support read receipts (delivered/seen status).
7. System should show list of chats with pagination.

Group Management

1. User should be able to create a group.
2. Admin should be able to add/remove members.
3. Admin should be able to delete group.

Non-functional Requirements

· **Scalability**

1. Target Users: 1 Billion users
2. Avg 100 messages per day per user
3. Total ≈ 100 Billion messages/day
4. Estimated Storage: ~100 TB (initial assumption)

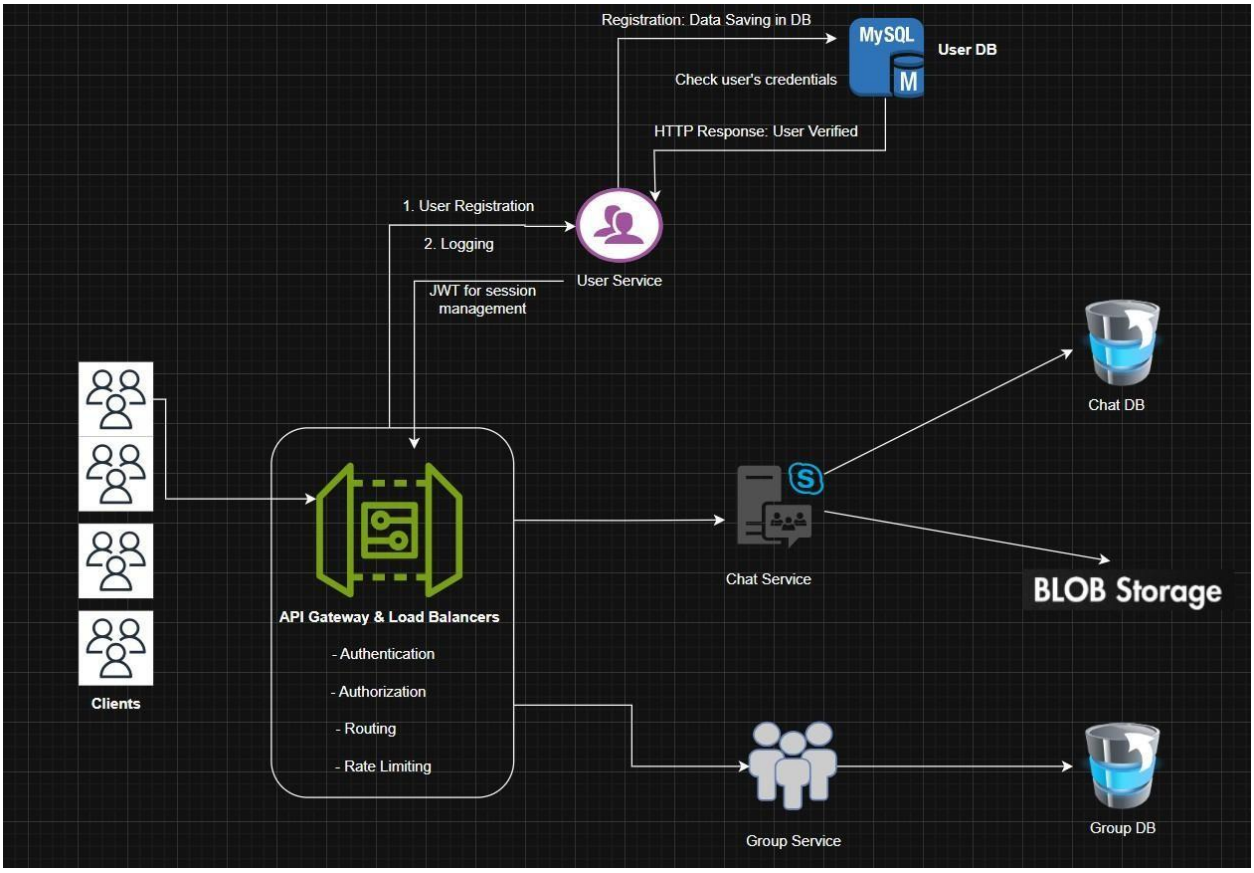
· **Availability (CAP Theorem)**
1. System should prefer **High Availability** over strict consistency.
2. Follow **Eventual Consistency** model.

· **Latency**
1. Response time: 50–80 ms.
2. Zero or negligible buffering.

· **Security**
1. JWT-based authentication.
2. Data encryption at rest and in transit (HTTPS + TLS).

· **High Availability**
1. Load balancing and redundancy.

Required System Design-



Outcome / Result -

After completing this experiment, we successfully designed and analyzed a scalable real- time messenger application with clearly defined functional and non-functional requirements, including user management, one-to-one and group messaging, media sharing, and read receipts. The system architecture was structured using REST APIs, WebSocket communication, NoSQL database design, and distributed system principles such as high availability and eventual consistency. The proposed design is capable of handling large-scale traffic with low latency and high reliability, making it suitable for real-world deployment similar to applications like WhatsApp.